

Московский государственный университет имени М.В. Ломоносова

Факультет вычислительной математики и кибернетики

Кафедра оптимального управления

Семестровое задание

**Отчёт о проделанной работе по практикуму
Универсальный решатель краевых задач (BVP)
с методом продолжения по параметру
на Python / PyQt5**

Составил: Ли Хунюй, группа 313

Руководитель: преподаватель Аввакумов С.Н.

Москва, 2026

ОГЛАВЛЕНИЕ

Введение	3
Актуальность	3
Цель и задачи работы	3
Объект и предмет исследования	4
1 Постановка задачи	5
2 Решение	5
3 Ввод и вывод	5
4 Обработка информации	6
5 Метод стрельбы	7
5.1 Уравнение	7
5.2 Алгоритм	7
5.3 Физический / вычислительный смысл	7
6 Метод продолжения по параметру	7
6.1 Уравнение	7
6.2 Задача Коши для продолжения	7
6.3 Совместное интегрирование состояния и вариаций	8
6.4 Аналогия	8
7 Инструментальные средства и программное окружение	8
7.1 Язык программирования и окружение	9
7.2 Научные библиотеки	9
7.3 Графический интерфейс	10
7.4 Система контроля версий	10
7.5 Среда разработки	10
8 Архитектура программы	10
8.1 Dataset — централизованное хранение	10
8.2 SymPyParser — безопасный парсинг	10
8.3 BVPSolver — вычислительное ядро	11
8.4 SolverWorker — многопоточность	11
8.5 BvpSolverApp — главное окно	11
8.6 IntegratedPlotWidget — интерактивная визуализация	11
9 Примеры расчётов	11
9.1 Пример 26.1 — краевая задача двух тел	12
9.2 Пример 26.2 — предельные циклы в системе Эквейлера	12
9.3 Пример 26.4 — задача быстрогодействия (лунка)	12
10 Заключение	12
11 Скриншоты программы	13

Введение

Актуальность

Нелинейные краевые задачи для систем обыкновенных дифференциальных уравнений (ОДУ) широко возникают в теории оптимального управления, механике, физике и инженерных приложениях. В частности, задача принципа максимума Понтрягина сводится к краевой задаче для совместной системы фазовых и сопряжённых уравнений, часто содержащей разрывные или нелинейные функции типа сигнатуры (sign), насыщения (sat) и мёртвой зоны (dez).

Классический метод стрельбы (shooting method) обладает квадратичной скоростью сходимости, но требует “достаточно хорошего” начального приближения и невырожденности матрицы Якоби вдоль итерационного процесса. Для сложных задач, возникающих в оптимальном управлении, эти требования часто не выполняются, что делает необходимым применение более робастных методов.

Метод продолжения по параметру (parameter continuation, homotopy method), развитый в работах кафедры оптимального управления ВМиК МГУ, позволяет преодолеть указанные недостатки, обеспечивая устойчивый поиск решения при грубом начальном приближении. Вместе с тем, существующие реализации данного метода (например, в среде Maple) требуют лицензионного ПО и не всегда обладают современным интерактивным интерфейсом.

Цель и задачи работы

Цель работы: разработка универсального программного комплекса для численного решения нелинейных краевых задач на языке Python с графическим интерфейсом пользователя (GUI) на базе библиотеки PyQt5, реализующего метод продолжения по параметру и метод стрельбы.

Задачи:

1. Изучить теоретические основы метода продолжения по параметру (разд. 7.25–7.26 учебного пособия [1]) и метода стрельбы.
2. Разработать архитектуру программного комплекса, включающую символьный парсер уравнений, вычислительное ядро и интерактивную визуализацию.
3. Реализовать безопасный парсинг математических выражений с помощью SymPy и численное интегрирование через SciPy.
4. Создать графический интерфейс, поддерживающий ввод уравнений, граничных условий, начальных приближений, а также выбор метода решения и параметров интегрирования.
5. Провести тестирование программы на классических примерах из учебного пособия [1]: краевая задача двух тел (пример 26.1), предельные циклы в системе Эквейлера (пример 26.2), задача быстрогодействия с областью управления в форме лунки (пример 26.4).
6. Реализовать экспорт результатов в формате JSON и интерактивное построение графиков решений и фазовых портретов.

Объект и предмет исследования

Объект исследования: нелинейные краевые задачи для систем обыкновенных дифференциальных уравнений, возникающие в задачах оптимального управления.

Предмет исследования: численные методы их решения, в частности метод продолжения по параметру и метод стрельбы, а также программная реализация указанных методов в виде интерактивного приложения.

1. Постановка задачи

Передо мной была поставлена задача создать небольшое приложение для численного решения нелинейных краевых задач для систем обыкновенных дифференциальных уравнений с использованием языка Python и библиотек SciPy, SymPy и PyQt5.

В качестве основного вычислительного метода был выбран **метод продолжения по параметру** (parameter continuation / homotopy method), описанный в разделах 7.25–7.26 учебного пособия [1], а также классический **метод стрельбы** (shooting method) для сравнения и как резервный вариант.

Я подготовил полноценный графический интерфейс (GUI) на PyQt5, позволяющий:

- вводить системы ОДУ в символьном виде;
- задавать граничные условия в универсальном формате;
- выбирать начальные приближения для неизвестных параметров;
- визуализировать решения в виде графиков $x_i(t)$ и фазовых портретов;
- экспортировать результаты в формате JSON.

2. Решение

Пользовательский интерфейс: интерфейс состоит из трёх основных зон:

1. *Библиотека задач* (левая панель) — таблица сохранённых задач с возможностью добавления, удаления, загрузки из JSON.
2. *Редактор уравнений* (центральная панель) — поля для ввода правых частей ОДУ, граничных условий, начальных значений и имён переменных.
3. *Панель параметров и управления* (правая панель) — настройка времени T , точности ε , метода интегрирования, выбор решателя (continuation / shooting), начальное приближение p_0 , кнопки запуска, построения графиков и экспорта.

Интерфейс позволяет удобно работать с различными краевыми задачами — в реализации предусмотрена валидация входных данных, многопоточное выполнение вычислений (через `QThread`) и встроенная система локализации (русский / китайский языки).

3. Ввод и вывод

Для ввода системы ОДУ предоставлено окно ввода уравнений вида

$$\dot{x}_i = f_i(t, x_0, x_1, \dots, x_{n-1}), \quad i = 0, \dots, n-1.$$

При наведении курсора возникают подсказки необходимого формата. Переменные состояния автоматически именуются как `x0`, `x1`, ... или задаются пользователем явно.

Граничные условия вводятся в форме векторного уравнения

$$R(x(t_*), x(T)) = 0,$$

где переменные обозначаются как $x0_0$, $x1_0$, ..., $x0_T$, $x1_T$,

Начальные условия разделяются на:

- *известные* (known) — задаются числом, фиксируются при решении;
- *неизвестные* (unknown) — составляют вектор параметров p , который определяется в процессе решения.

На выход можно получить:

- численное решение $x(t)$ на отрезке $[t_*, T]$;
- вектор найденных начальных параметров p_{opt} ;
- норму невязки $\|\Phi\|$ и число итераций;
- графики решений и фазовые портреты с интерактивными настройками (сетка, маркеры S/F, равные пропорции, толщина линий);
- экспорт в JSON / TXT.

Кнопка **Clear** позволяет очистить все поля без перезапуска программы.

4. Обработка информации

Обработка полученных данных происходит с учётом правил вычислительной математики:

1. **Символьный парсинг.** С помощью SymPy строки уравнений преобразуются в аналитические выражения, строится якобиан $\partial f / \partial x$. Это позволяет автоматически генерировать NumPy-функции для правых частей и матрицы вариаций.
2. **Валидация.** Перед запуском проверяется согласованность размерностей: число уравнений, число граничных условий, число неизвестных начальных значений и длина вектора p_0 .
3. **Интегрирование.** Внутренняя задача Коши решается методами RK45, DOP853, Radau, BDF из `scipy.integrate.solve_ivp` с заданными абсолютной и относительной точностями.
4. **Построение невязки.** Для заданного p выполняется интегрирование от t_* до T , затем вычисляется вектор невязки граничных условий $\Phi(p) = R(x(t_*, p), x(T, p))$.
5. **Численное дифференцирование (резервный метод).** Если совместное интегрирование с вариационным уравнением не сходится, якобиан $\Phi'(p)$ вычисляется численно с адаптивным шагом.

5. Метод стрельбы

5.1. Уравнение

Краевая задача сводится к конечному нелинейному векторному уравнению

$$\Phi(p) = 0, \quad \Phi : \mathbb{R}^n \rightarrow \mathbb{R}^n, \quad (1)$$

где p — вектор неизвестных начальных условий. Классический метод Ньютона

$$p_{k+1} = p_k - [\Phi'(p_k)]^{-1} \Phi(p_k), \quad k = 0, 1, \dots \quad (2)$$

требует невырожденности матрицы $\Phi'(p)$ вдоль процесса и “достаточно хорошего” начального приближения p_0 .

5.2. Алгоритм

В программе реализована двухуровневая стратегия:

1. `scipy.optimize.root` с методом `hybr` — быстрый, хорошо работает при качественном p_0 ;
2. `scipy.optimize.least_squares` с методом Левенберга–Марквардта (LM) — более робастный, если `hybr` не сходится.

5.3. Физический / вычислительный смысл

Метод стрельбы интерпретирует краевую задачу как “пристрелку”: подбирая начальные параметры p , мы “попадаем” в граничные условия на правом конце $t = T$. Недостаток — сходимость сильно зависит от начального приближения.

6. Метод продолжения по параметру

6.1. Уравнение

Для преодоления недостатка метода Ньютона используется **метод продолжения по параметру** [1, § 7.25]. Вспомогательное уравнение имеет вид:

$$\Phi(p) = (1 - \mu) \Phi(p_0), \quad \mu \in [0, 1], \quad (3)$$

где p_0 — произвольная фиксированная точка (начальное приближение), погрешность которой *не предполагается малой*. При $\mu = 0$ решение известно: $p(0) = p_0$; при $\mu = 1$ уравнение (3) совпадает с исходным (1).

6.2. Задача Коши для продолжения

Предполагая существование гладкой ветви $p = p(\mu)$ и невырожденность $\Phi'(p(\mu))$, дифференцирование (3) по μ даёт

$$\Phi'(p(\mu)) \frac{dp}{d\mu} = -\Phi(p_0). \quad (4)$$

Отсюда получаем задачу Коши (внешняя задача):

$$\frac{dp}{d\mu} = -[\Phi'(p)]^{-1}\Phi(p_0), \quad p(0) = p_0, \quad 0 \leq \mu \leq 1. \quad (5)$$

Численное решение задачи Коши (5) методом Эйлера с шагом $\Delta\mu = 1$ превращается в один шаг метода Ньютона (2). В программе используется *многошаговая* схема: отрезок $[0, 1]$ разбивается на N частей, на каждом шаге $\mu_k = k/N$ промежуточная задача

$$\Phi(p) = (1 - \mu_k)\Phi(p_0)$$

решается методом Ньютона с демпфированием, стартуя с решения предыдущего шага. Это позволяет устойчиво следовать вдоль выбранной ветви решения.

6.3. Совместное интегрирование состояния и вариаций

Для одновременного вычисления $x(t, p)$ и матрицы вариаций

$$X(t, p) \equiv \frac{\partial x(t, p)}{\partial p}$$

решается внутренняя векторно-матричная задача Коши [1, § 7.26]:

$$\begin{cases} \dot{x} = f(t, x), & x|_{t=t_*} = p, \\ \dot{X} = f'_x(t, x) X, & X|_{t=t_*} = I, \end{cases} \quad a \leq t \leq b. \quad (6)$$

Матрица $\Phi'(p)$ выражается через $X(a, p)$ и $X(b, p)$:

$$\Phi'(p) = R'_x X(a, p) + R'_y X(b, p).$$

В программе это реализовано как `combined_ode` (совместное интегрирование), с резервным переходом на чистое численное дифференцирование при вырожденности.

6.4. Аналогия

Метод продолжения можно рассматривать как **гомотопию** (деформацию) от тривиальной задачи к исходной. Параметр μ играет роль “времени” в пространстве параметров: при $\mu = 0$ мы находимся в “известной” точке p_0 , а при $\mu = 1$ “достигаем” искомого решения, непрерывно двигаясь по гладкой ветви.

7. Инструментальные средства и программное окружение

При разработке программного комплекса использовался стек технологий, рекомендованный в методических материалах кафедры [8], а также стандартные инструменты научного программирования на Python [5, 6, 7].

7.1. Язык программирования и окружение

Python 3.10+ — основной язык разработки. Выбор обусловлен богатой экосистемой библиотек для научных вычислений, простотой синтаксиса и кросс-платформенностью. Для изоляции зависимостей использовалось стандартное виртуальное окружение `venv`:

```
1 python -m venv venv
2 # Windows: venv\Scripts\activate
3 # Linux/macOS: source venv/bin/activate
```

Листинг 1: Создание виртуального окружения

Управление зависимостями осуществлялось через файл `requirements.txt`, содержащий перечень необходимых пакетов с указанием версий (см. табл. 1).

Пакет	Версия	Назначение
numpy	≥ 1.21	Массивы, линейная алгебра, векторизация
scipy	≥ 1.9	Численное интегрирование (<code>solve_ivp</code>), оптимизация
sympy	≥ 1.11	Символьные вычисления, парсинг, <code>lambdify</code>
matplotlib	≥ 3.5	Визуализация данных и фазовых портретов
PyQt5	≥ 5.15	Графический интерфейс пользователя (GUI)

Таблица 1 – Основные зависимости проекта

7.2. Научные библиотеки

NumPy [5] — фундаментальный пакет для работы с многомерными массивами. В проекте используется для представления векторов состояния $x(t)$, матриц вариаций $X(t, p)$ и векторно-матричных операций в методе продолжения.

SciPy [6] — расширение NumPy, содержащее библиотеки для интегрирования и оптимизации. Ключевые компоненты:

- `scipy.integrate.solve_ivp` — решение задачи Коши методами Рунге–Кутты (RK45, DOP853) и жёстких систем (Radau, BDF);
- `scipy.optimize.root` (метод `hybr`) и `scipy.optimize.least_squares` (метод `lm`) — реализация метода стрельбы.

SymPy [7] — библиотека символьной математики. Используется для:

- безопасного парсинга строк уравнений через `sympy.parsing.parse_expr` (вместо небезопасного `eval`);
- автоматического построения якобиана $\partial f_i / \partial x_j$;
- преобразования символьных выражений в NumPy-функции через `lambdify`.

Matplotlib — библиотека визуализации. В проекте используется совместно с PyQt5 через backend `Qt5Agg` для встраивания интерактивных графиков в окно приложения.

7.3. Графический интерфейс

PyQt5 — Python-обёртка над фреймворком Qt. Обеспечивает кросс-платформенный GUI с богатым набором виджетов (`QTableWidget`, `QLineEdit`, `QTextEdit`, `QComboBox` и др.). Для выполнения длительных вычислений без блокировки интерфейса используется механизм сигналов и слотов в сочетании с `QThread` [8].

7.4. Система контроля версий

Git + GitHub — система контроля версий и удалённый репозиторий. В рамках проекта реализованы:

- регулярные коммиты (не реже одного раза в две недели);
- осмысленные сообщения коммитов на английском языке;
- файл `README.md` с описанием структуры и инструкцией по запуску;
- файл `requirements.txt` со списком зависимостей.

7.5. Среда разработки

Visual Studio Code (VS Code) — основная интегрированная среда разработки. Используемые расширения: Python (Microsoft), Pylance, GitLens. Настроен запуск и отладка Python-скриптов с активацией виртуального окружения.

8. Архитектура программы

Программа построена по модульному принципу. Основные компоненты:

8.1. Dataset — централизованное хранение

Класс `Dataset` (`dataclass`) хранит все параметры задачи:

- `equations` — список строк правых частей ОДУ;
- `boundary_conditions` — список строк для $R[x(t_*), x(T)] = 0$;
- `initial_values` — словарь известных/неизвестных начальных значений;
- `guess` — начальное приближение p_0 ;
- `known_indices` / `unknown_indices` — разбиение переменных;
- параметры интегрирования: T , ε , метод, число шагов продолжения.

8.2. SymPyParser — безопасный парсинг

С помощью `sympy.parsing.parse_expr` строки уравнений преобразуются в SymPy-выражения. Реализован “белый список” разрешённых имён (`sin`, `cos`, `exp`, `log`, `sqrt`, ...). Якобиан $\partial f_i / \partial x_j$ строится автоматически. Функции `lambdify` порождают NumPy-callable объекты.

8.3. BVPSolver — вычислительное ядро

Реализует два метода:

- `solve_shooting()` — решение $\Phi(p) = 0$ через `scipy.optimize`;
- `solve_continuation()` — параметрическая гомотопия с демпфированным методом Ньютона на каждом шаге μ_k .

Также содержит методы численного дифференцирования $\Phi'(p)$ как резервный вариант.

8.4. SolverWorker — многопоточность

Наследник `QThread`. Вычисления выполняются в отдельном потоке, не блокируя GUI. Испускаются сигналы `progress` (метод, процент, сообщение), `finished` (результат) и `error`.

8.5. BvpSolverApp — главное окно

Класс `QMainWindow`, объединяющий:

- таблицу задач (библиотека);
- редактор уравнений (динамическая генерация полей по размерности n);
- панель параметров;
- индикатор прогресса;
- меню локализации и темы.

8.6. IntegratedPlotWidget — интерактивная визуализация

Встроенный виджет с `matplotlib`-канвасом и панелью управления:

- выбор оси X и нескольких осей Y (наложение кривых);
- переключение стилей линий (все сплошные / вспомогательные пунктир);
- флажки: сетка, маркеры S/F, равные пропорции;
- список решений с цветовой индикацией и возможностью скрытия.

9. Примеры расчётов

В программе предусмотрена загрузка задач из JSON. Ниже приведены примеры, соответствующие задачам из учебного пособия [1].

9.1. Пример 26.1 — краевая задача двух тел

Система:

$$\begin{cases} \dot{x}_1 = x_3, & x_1(0) = 2, \quad x_1(7) = 1.0738644361, \\ \dot{x}_2 = x_4, & x_2(0) = 0, \quad x_2(7) = -1.0995343576, \\ \dot{x}_3 = -x_1(x_1^2 + x_2^2)^{-3/2}, \\ \dot{x}_4 = -x_2(x_1^2 + x_2^2)^{-3/2}. \end{cases}$$

При выборе $t_* = 0$ и начальных приближений $p_{01} = [2, 0, -0.5, 0.5]$ и $p_{02} = [2, 0, 0.5, -0.5]$ получены два различных решения, что демонстрирует способность метода продолжения находить разные ветви.

9.2. Пример 26.2 — предельные циклы в системе Эквейлера

Система:

$$\begin{cases} \dot{x}_1 = x_3 x_2, & x_1(0) = x_4(0), \quad x_1(1) = x_4(1), \\ \dot{x}_2 = x_3(-x_1 + \sin x_2), & x_2(0) = 0, \quad x_2(1) = 0, \\ \dot{x}_3 = 0, \\ \dot{x}_4 = 0. \end{cases}$$

Здесь $x_3 = T$ — неизвестный период, $x_4 = x_1(0)$. Выбор различных p_0 позволяет находить различные предельные циклы (см. рис. 26.2 в [1]).

9.3. Пример 26.4 — задача быстрогодействия (лунка)

Система с гладкой аппроксимацией области управления U_μ (лунка):

$$\begin{cases} \dot{x}_1 = T(x_2 + c'_{\psi_1}(U_\mu, \psi)), & x_1(0) = 4, \quad x_1(1) = 0, \\ \dot{x}_2 = T(-1.5x_1 - 0.25x_2 + c'_{\psi_2}(U_\mu, \psi)), & x_2(0) = 1, \quad x_2(1) = 0, \\ \dot{\psi}_1 = -T\psi_2, & \psi_1^2(1) + \psi_2^2(1) = 1, \\ \dot{\psi}_2 = T(1.5\psi_1 + 0.25\psi_2), \\ \dot{T} = 0. \end{cases}$$

Программа корректно обрабатывает вспомогательные переменные u_1, u_2 , вычисляемые из ОДУ обратно (см. метод `_compute_aux_outputs` в коде).

10. Заключение

Каждый реализованный метод кодирует определённый тип вычислительного поведения:

Компонент	Метод / Аналогия	Тип поведения
Shooting	Метод Ньютона	Локальная квадратичная сходимость
Continuation	Гомотопия по μ	Глобальная устойчивость, робастность
SymPy Parser	Символьная алгебра	Универсальность, безопасность
PyQt5 GUI	Интерактивная среда	Наглядность, доступность
Multi-threading	Параллельные вычисления	Отзывчивость интерфейса

Таблица 2 – Сравнительная характеристика компонентов программы

Разработанный решатель позволяет:

- решать регулярные краевые задачи для ОДУ;
- исследовать задачи принципа максимума Понтрягина (включая задачи со сглаженными нелинейностями типа **sat**, **sign**);
- находить периодические решения и предельные циклы;
- визуально анализировать поведение фазовых траекторий.

11. Скриншоты программы

Ниже представлены основные экраны приложения.

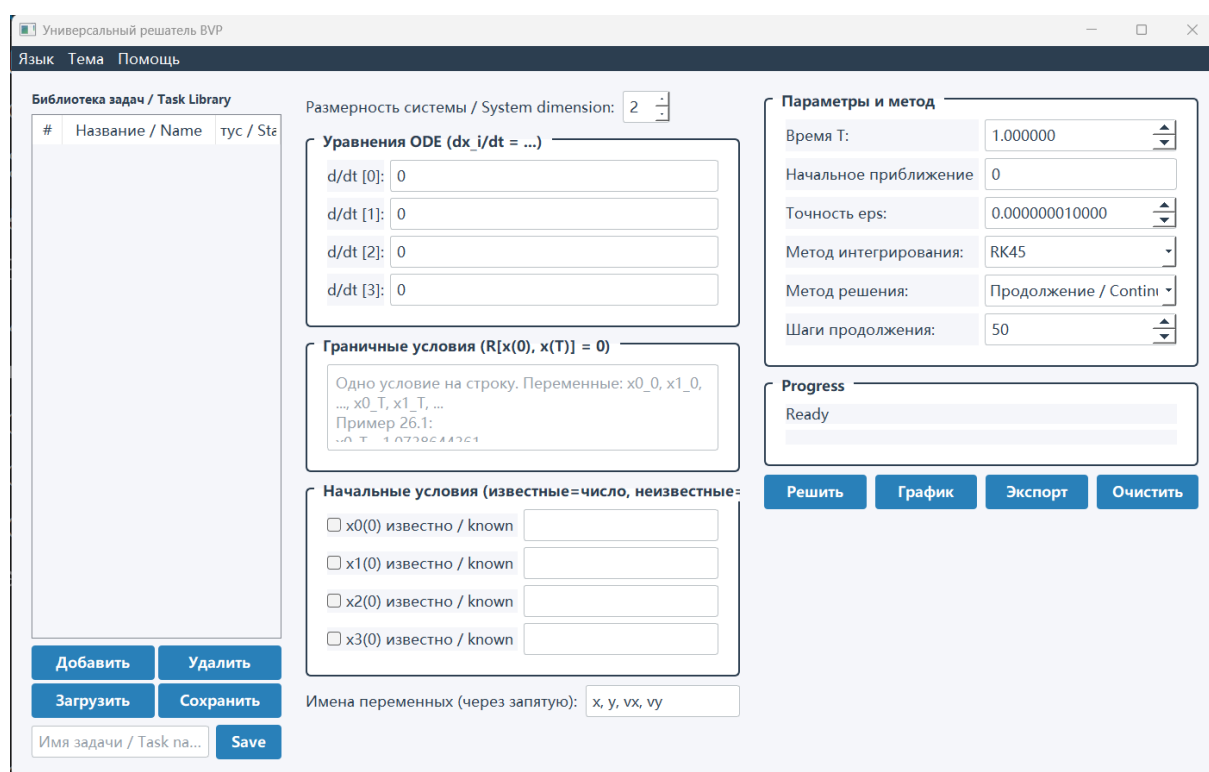


Рис. 1 – Общий вид интерфейса (русский)



Рис. 2 – Общий вид интерфейса (китайский)



Рис. 3 – Пример 26.1

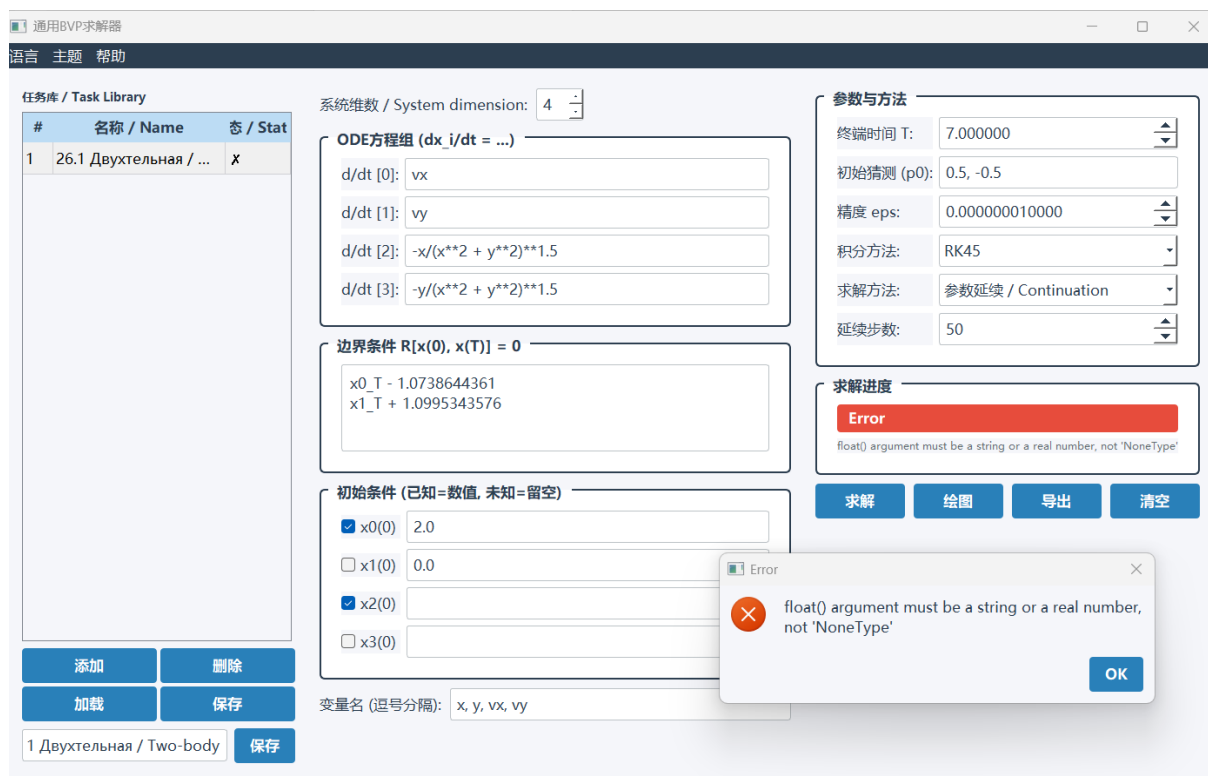


Рис. 4 – Окно сообщения об ошибке



Рис. 5 – Окно информации об авторе

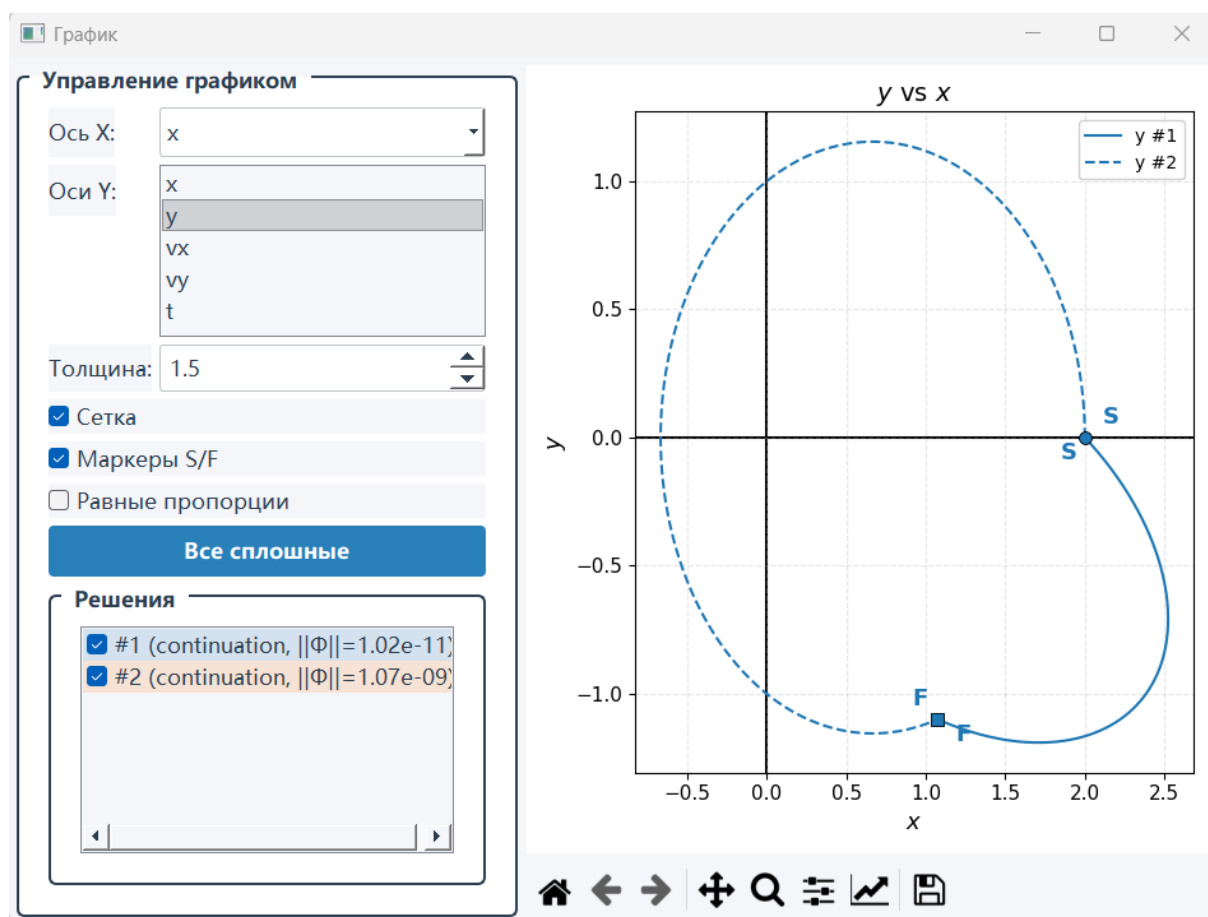


Рис. 6 – Результат работы 26.1

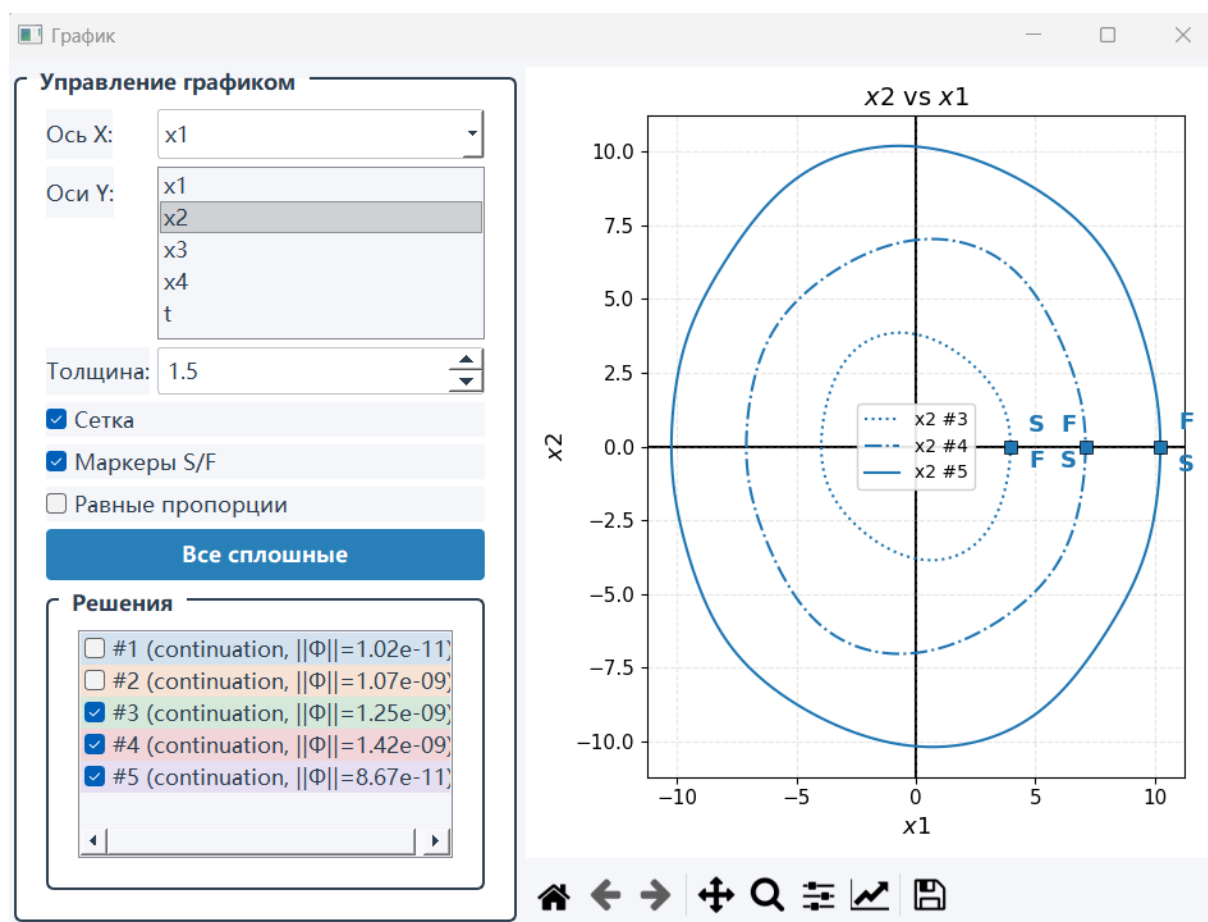


Рис. 7 – Результат работы 26.2

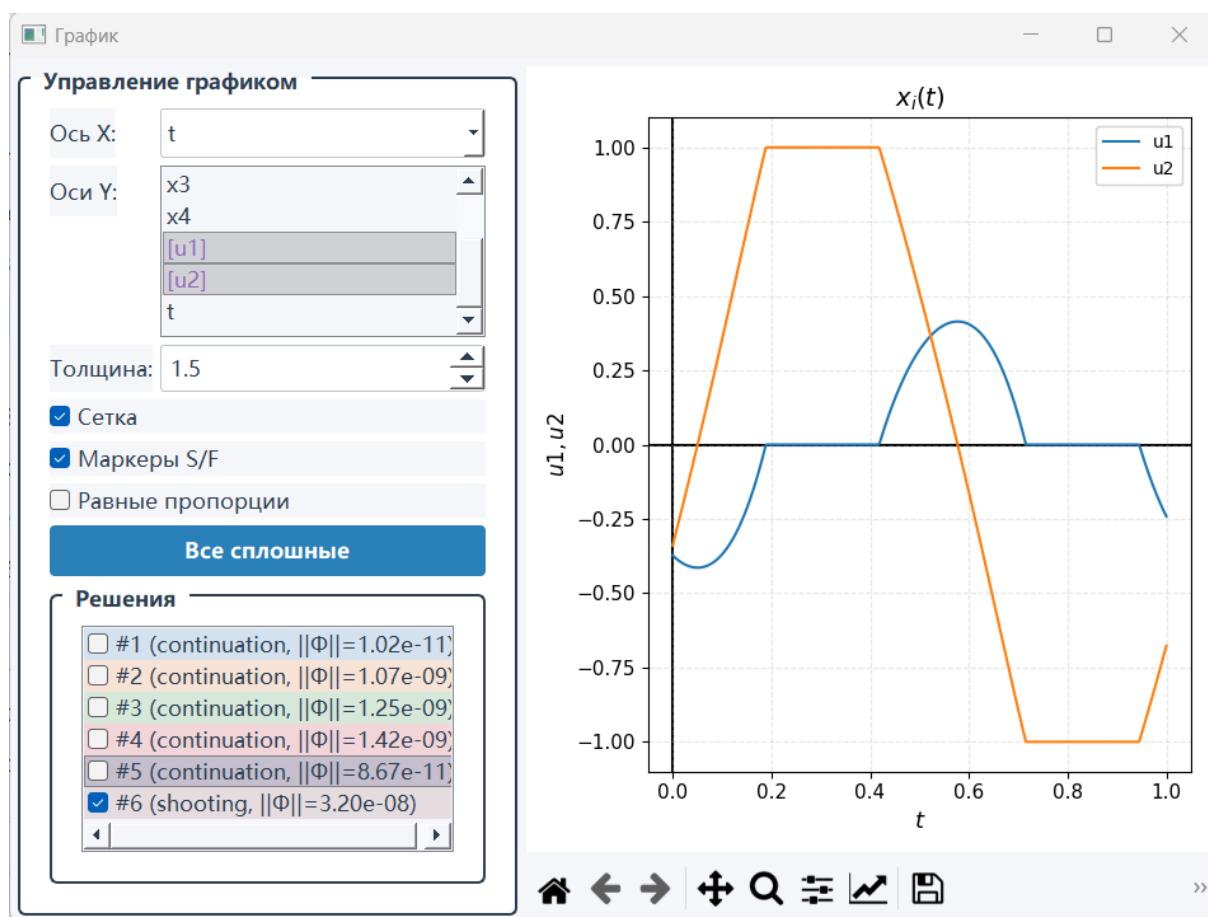


Рис. 8 – Результат работы 26.4

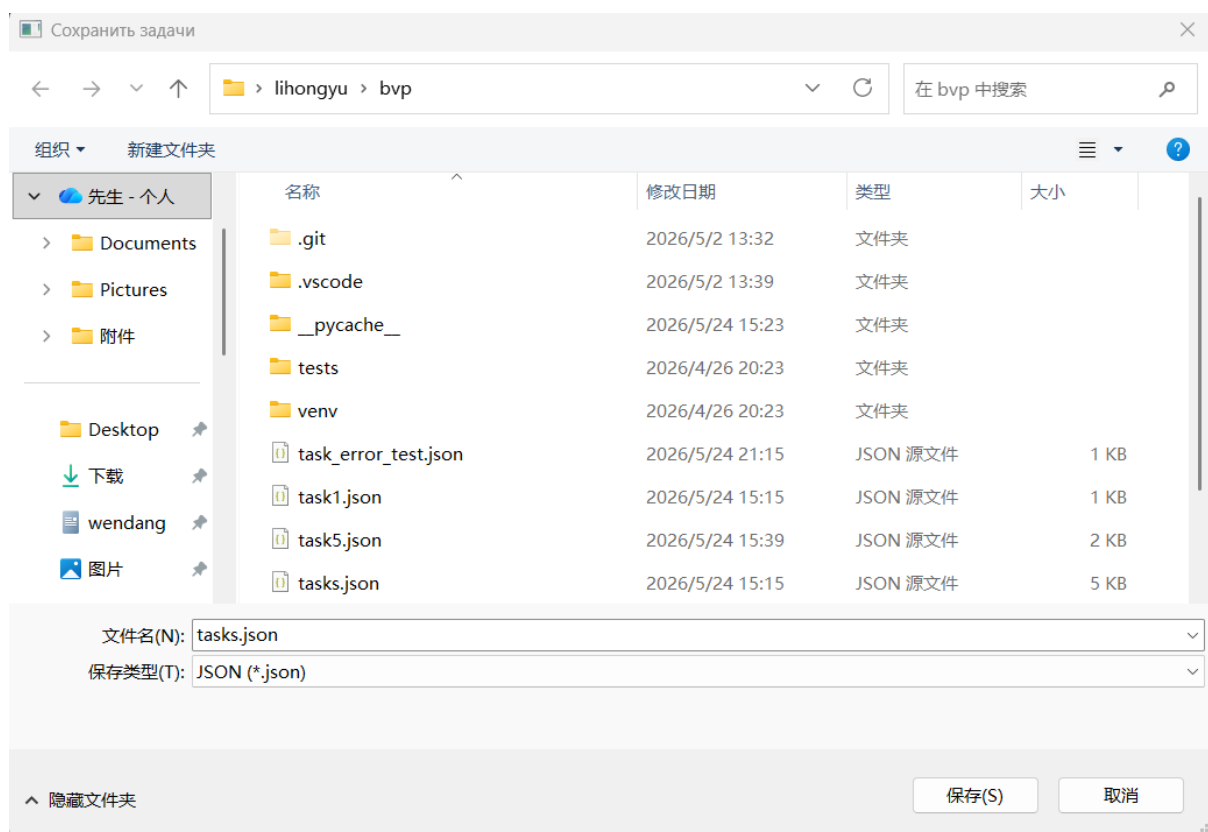


Рис. 9 – Окно сохранения

Список использованных источников и литературы

1. Киселёв Ю.Н., Аввакумов С.Н., Орлов М.В. Оптимальное управление. Линейная теория и приложения: учеб. пособие для студентов фак. ВМиК МГУ / Ю.Н. Киселёв, С.Н. Аввакумов, М.В. Орлов. — М.: Издат. отд. фак. ВМиК МГУ им. М.В. Ломоносова, 2007. — 304 с.
2. Риверз Б. PyQt5. Разработка приложений: практикум / Б. Риверз. — СПб.: Символ-Плюс, 2021. — 352 с.
3. Орлов С.М. Метод продолжения по параметру на Python: презентация лекции / С.М. Орлов. — М.: Кафедра оптимального управления ВМиК МГУ, 2026. — 29 с.